

DOCUMENTO DE MODELO CONCEPTUAL

Sistema de Gestión de Tickets

Autor: Miguel Mancilla Mercado

Fecha: 02/03/2026

ÍNDICE

1. Introducción
2. Alcance del Sistema
3. Entidades del Modelo Conceptual
4. Relaciones
5. Reglas de Negocio
6. Consideraciones Arquitectónicas

1. Introducción

El presente documento describe el modelo conceptual del Sistema de Gestión de Tickets. Este documento define las entidades principales, sus relaciones y las reglas de negocio que rigen el funcionamiento del sistema antes de su implementación técnica.

2. Alcance del Sistema

El sistema permitirá la creación, gestión y seguimiento de tickets de soporte interno. Los usuarios podrán crear tickets, comentar y visualizar su estado. El personal de soporte podrá gestionar, asignar y cerrar tickets.

3. Entidades del Modelo Conceptual

Usuario: Representa a los empleados, soporte y administradores del sistema.

Ticket: Representa una solicitud o incidencia creada por un usuario.

Comentario: Permite agregar información adicional a un ticket.

HistorialCambio: Registra modificaciones relevantes realizadas a un ticket.

Área: Agrupa usuarios según su unidad organizacional.

Prioridad: Clasifica el nivel de urgencia de un ticket.

4. Relaciones

Un Usuario puede crear uno o más Tickets (1:N).

Un Ticket pertenece a un Usuario creador.

Un Ticket puede estar asignado a un Usuario (soporte).

Un Ticket puede tener múltiples Comentarios (1:N).

Un Ticket puede tener múltiples registros en HistorialCambio (1:N).

Un Usuario pertenece a un Área (N:1).

Un Ticket posee una Prioridad (N:1).

5. Reglas de Negocio

RN-01: Un ticket no puede reabrirse si han pasado más de 7 días desde su fecha de cierre. En este caso, el sistema deberá generar un nuevo ticket.

RN-02: Todo ticket creado debe tener como estado inicial 'ABIERTO' de manera obligatoria.

RN-03: Todas las modificaciones relevantes realizadas a un ticket deben quedar registradas en el Historial de Cambios, incluyendo usuario y fecha.

6. Consideraciones Arquitectónicas

El modelo conceptual servirá como base para la implementación en Django utilizando Django REST Framework para la exposición de servicios API y PostgreSQL como motor de base de datos. La arquitectura seguirá el patrón cliente-servidor con separación entre frontend y backend.